Discrete Flow Matching

Reading Group

Andre He

March 9, 2026

Carnegie Mellon University

Papers Covered

1. **Discrete Flow Matching**
   I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Q. Chen, G. Synnaeve, Y. Adi, Y. Lipman (Meta / Weizmann), 2024.

2. **Generative Flows on Discrete State-Spaces: Enabling Multimodal Flows with Applications to Protein Co-Design**
   A. Campbell, J. Yim, R. Barzilay, T. Rainforth, T. Jaakkola (Oxford / MIT), 2024.

Outline

Recap: Continuous Flow Matching

Discrete Flow Matching

Comparison to Discrete Diffusion

Empirical Results

Recap: Continuous Flow Matching

Setup

Data space: $\mathbf{x} = (x_1, \ldots, x_d) \in \mathbb{R}^d$

Goal: Learn a generative model for a data distribution $q(\mathbf{x}_1)$.

Strategy (CNFs):

- Define a *probability path* $p_t : [0,1] \times \mathbb{R}^d \to \mathbb{R}_{>0}$ interpolating

$$p_0 = \mathcal{N}(0, I) \quad \longrightarrow \quad p_1 = q$$

- Learn a time-dependent *vector field* $v_t(\mathbf{x}; \theta)$ (neural network) that *generates* this path.
- At inference: integrate the ODE from noise to obtain a sample.

Flows and the Continuity Equation

A vector field v_t defines a **flow** ϕ_t via the ODE:

$$\frac{d}{dt}\phi_t(\mathbf{x}) = v_t(\phi_t(\mathbf{x})), \qquad \phi_0(\mathbf{x}) = \mathbf{x}$$

The flow reshapes a prior p_0 into $p_t = [\phi_t]_* p_0$ (push-forward).

How do we know v_t generates p_t?

The **continuity equation** gives a necessary and sufficient condition:

$$\frac{\partial}{\partial t} p_t(\mathbf{x}) + \operatorname{div}\big(p_t(\mathbf{x})\, v_t(\mathbf{x})\big) = 0$$

Intuition: probability mass is conserved as it flows along v_t.

Flow Matching

Given a target vector field u_t generating p_t, define the **Flow Matching (FM)** loss:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t,\, p_t(\mathbf{x})} \| v_t(\mathbf{x}) - u_t(\mathbf{x}) \|^2$$

Key problem: the *marginal* vector field u_t is **intractable** — we have no closed form for it in general.

Solution: Conditional Probability Paths

Idea: build the marginal path from simpler *conditional* paths.

For each data point $\mathbf{x}_1 \sim q$, define $p_t(\mathbf{x} \mid \mathbf{x}_1)$ such that:

$$p_0(\mathbf{x} \mid \mathbf{x}_1) = \mathcal{N}(\mathbf{x} \mid 0, I), \qquad p_1(\mathbf{x} \mid \mathbf{x}_1) = \delta_{\mathbf{x}_1}(\mathbf{x})$$

Marginalising over q gives a path with the right endpoints:

$$p_t(\mathbf{x}) = \int p_t(\mathbf{x} \mid \mathbf{x}_1)\, q(\mathbf{x}_1)\, d\mathbf{x}_1 \quad \Rightarrow \quad p_0 = \mathcal{N}(0, I), \quad p_1 = q$$

Similarly, define a **conditional VF** $u_t(\mathbf{x} \mid \mathbf{x}_1)$ that generates $p_t(\cdot \mid \mathbf{x}_1)$.

The marginal VF is then:

$$u_t(\mathbf{x}) = \int u_t(\mathbf{x} \mid \mathbf{x}_1)\, \frac{p_t(\mathbf{x} \mid \mathbf{x}_1)\, q(\mathbf{x}_1)}{p_t(\mathbf{x})}\, d\mathbf{x}_1$$

Key Theorem: Marginalisation of Vector Fields

Theorem 1 (Lipman et al., 2023)
Given conditional vector fields $u_t(\mathbf{x} \mid \mathbf{x}_1)$ that each generate $p_t(\cdot \mid \mathbf{x}_1)$, the **marginal** vector field

$$u_t(\mathbf{x}) = \int u_t(\mathbf{x} \mid \mathbf{x}_1) \, \frac{p_t(\mathbf{x} \mid \mathbf{x}_1) \, q(\mathbf{x}_1)}{p_t(\mathbf{x})} \, d\mathbf{x}_1$$

generates the marginal probability path $p_t(\mathbf{x})$.

Implication: we can freely choose conditional paths with *tractable* vector fields, then marginalise to obtain the VF that generates the marginal path — which by construction has $p_1 = q$.

Conditional Flow Matching (CFM) Objective

Key insight (Theorem 2): replace the intractable $u_t(\mathbf{x})$ with $u_t(\mathbf{x} \mid \mathbf{x}_1)$ in the loss:

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,\, q(\mathbf{x}_1),\, p_t(\mathbf{x}|\mathbf{x}_1)} \left\| v_t(\mathbf{x}) - u_t(\mathbf{x} \mid \mathbf{x}_1) \right\|^2$$

Why it works: $\mathcal{L}_{\mathrm{CFM}}$ and $\mathcal{L}_{\mathrm{FM}}$ differ only by a term that is constant in θ, so they have **identical gradients**. This follows because the cross term in the expanded square is the same under both expectations — conditioning on $\mathbf{x}_1$ and marginalising out gives back the marginal VF.

Summary: The CFM Recipe

1. **Choose a conditional path.** Pick $p_t(\mathbf{x} \mid \mathbf{x}_1)$ (e.g. OT or diffusion) with boundary conditions
$$p_0(\cdot \mid \mathbf{x}_1) = \mathcal{N}(0, I), \qquad p_1(\cdot \mid \mathbf{x}_1) = \delta_{\mathbf{x}_1}.$$
2. **Derive the conditional VF.** Compute $u_t(\mathbf{x} \mid \mathbf{x}_1)$ in closed form from the path (e.g. via Theorem 3).
3. **Train by regression.** For each mini-batch, sample $\mathbf{x}_1 \sim q$, $\mathbf{x}_0 \sim \mathcal{N}(0, I)$, $t \sim U[0, 1]$, and minimise
$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}\left\| v_t(\psi_t(\mathbf{x}_0); \theta) - u_t(\psi_t(\mathbf{x}_0) \mid \mathbf{x}_1) \right\|^2.$$
4. **Generate samples.** Draw $\mathbf{x}_0 \sim \mathcal{N}(0, I)$ and integrate the learned ODE
$$\dot{\mathbf{x}} = v_t(\mathbf{x}; \theta), \quad t \in [0, 1]$$
with any off-the-shelf ODE solver.

Discrete Flow Matching

Setup

Data space: $\mathbf{x} = (x^1, \ldots, x^N) \in \mathcal{D} = [V]^N$, where each token $x^i \in [V] = \{1, \ldots, V\}$.

Goal: transform source samples $X_0 \sim p$ into target samples $X_1 \sim q$.

Source (masked): add a special **mask** token $\mathtt{m}$ to the vocabulary and set

$$p(\mathbf{x}) = \delta_{\mathbf{m}}(\mathbf{x}) \qquad \Longleftrightarrow \qquad X_0 = (\mathtt{m}, \mathtt{m}, \ldots, \mathtt{m})$$

Target: the data distribution $q(\mathbf{x})$ (e.g. natural language), with data points $\mathbf{x}_1 \sim q$.

Conditional Probability Path

As in continuous FM, define a **probability path** p_t interpolating

$$p_0 = p \quad \longrightarrow \quad p_1 = q.$$

Build from conditional paths. For each data point $\mathbf{x}_1 \sim q$, define a per-token path

$$p_t(\mathbf{x} \mid \mathbf{x}_1) = \prod_{i=1}^{N} p_t(x^i \mid \mathbf{x}_1), \qquad p_t(x^i \mid \mathbf{x}_1) = (1 - \kappa_t)\, \delta_{\mathtt{m}}(x^i) + \kappa_t\, \delta_{x_1^i}(x^i),$$

with scheduler $\kappa_0 = 0$, $\kappa_1 = 1$, κ_t monotonically increasing.

Interpretation: at time t, token i equals the *target* x_1^i with probability κ_t and $\mathtt{m}$ with probability $1 - \kappa_t$.

Marginal Probability Path

Marginal path (marginalise over q):

$$p_t(\mathbf{x}) = \sum_{\mathbf{x}_1 \in \mathcal{D}} \prod_{i=1}^{N} p_t(x^i \mid \mathbf{x}_1)\; q(\mathbf{x}_1).$$

Check the endpoints:

At $t = 0$: $\kappa_0 = 0$, so $p_0(x^i \mid \mathbf{x}_1) = \delta_{\mathtt{m}}(x^i)$ for all $\mathbf{x}_1$. Hence

$$p_0(\mathbf{x}) = \sum_{\mathbf{x}_1} \prod_{i} \delta_{\mathtt{m}}(x^i)\; q(\mathbf{x}_1) = \delta_{\mathbf{m}}(\mathbf{x}) = p(\mathbf{x}). \quad \checkmark$$

At $t = 1$: $\kappa_1 = 1$, so $p_1(x^i \mid \mathbf{x}_1) = \delta_{x_1^i}(x^i)$. Hence

$$p_1(\mathbf{x}) = \sum_{\mathbf{x}_1} \prod_{i} \delta_{x_1^i}(x^i)\; q(\mathbf{x}_1) = \sum_{\mathbf{x}_1} \delta_{\mathbf{x}_1}(\mathbf{x})\; q(\mathbf{x}_1) = q(\mathbf{x}). \quad \checkmark$$

Strategy: Model the Path as a CTMC

In continuous FM, the probability path is generated by an ODE (a deterministic flow).

In the discrete setting, there is no continuous space to "flow" through. Instead, we model the probability path using a **Continuous-Time Markov Chain (CTMC)**: a stochastic process where a discrete state $x_t \in [V]$ jumps between values in continuous time.

Background: CTMCs

A CTMC on state space $[V]$ is governed by a **rate matrix** $R_t \in \mathbb{R}^{V \times V}$:

- $R_t(x, y) \geq 0$ for $x \neq y$ (transition rates are non-negative),
- $R_t(x, x) = -\sum_{y \neq x} R_t(x, y)$ (rows sum to zero).

In an infinitesimal step h, the transition probability is

$$p_{t+h|t}(y \mid x_t) = \delta_{x_t}(y) + R_t(x_t, y)\, h.$$

Interpretation: with probability $R_t(x_t, y)\, h$ the chain jumps from x_t to $y \neq x_t$; otherwise it stays put.

Rate Matrices as Velocities

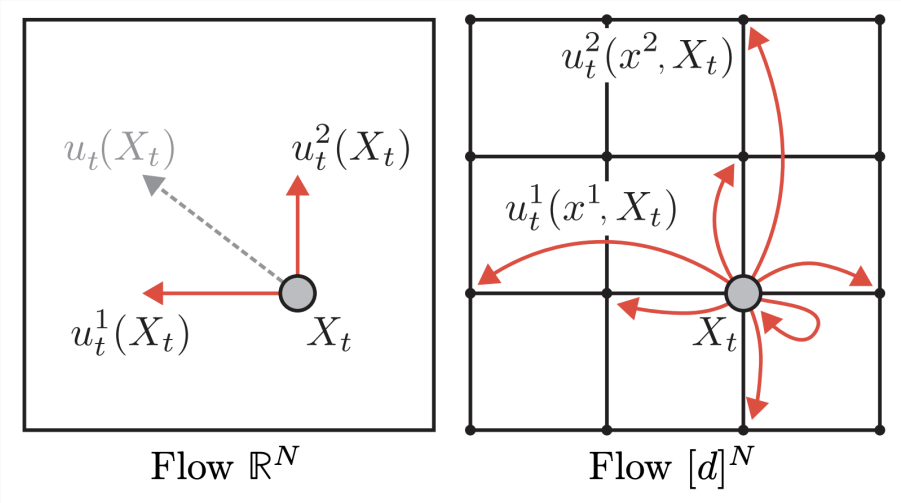
Continuous FM: each coordinate updated deterministically via a velocity field,

$$X^i_{t+h} = X^i_t + h\, u^i_t(X_t).$$

Discrete FM: each token is updated independently as a CTMC Euler step:

$$\boxed{X^i_{t+h} \sim \delta_{X^i_t}(\cdot) + h\, R^i_t(\cdot,\, X_t)}$$

Rate Matrices as Velocities (Figure)



Source: Gat et al., 2024

The Discrete Continuity Equation

How do we know rate matrices R_t generate a probability path p_t?

For continuous flows, we used the **continuity equation**:

$$\frac{\partial}{\partial t} p_t(\mathbf{x}) = -\operatorname{div}\big(p_t(\mathbf{x})\, v_t(\mathbf{x})\big).$$

The discrete analogue is the **Kolmogorov equation**:

$$\boxed{\frac{\partial}{\partial t} p_t(\mathbf{x}) = \sum_{\mathbf{z} \in \mathcal{D}} R_t(\mathbf{z}, \mathbf{x})\, p_t(\mathbf{z})}$$

Intuition: the rate of change of $p_t(\mathbf{x})$ is the incoming flux from other states minus the outgoing flux from $\mathbf{x}$ (the $R_t(\mathbf{x}, \mathbf{x})\, p_t(\mathbf{x})$ term, since $R_t(\mathbf{x}, \mathbf{x}) < 0$).

More on the Kolmogorov Equation

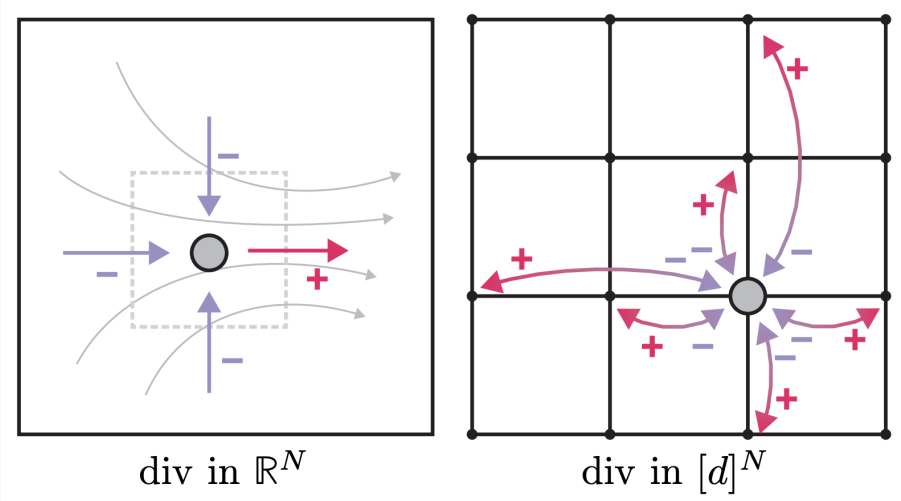
The Kolmogorov equation is really a **discrete continuity equation**. Define a **discrete divergence** for a flux $v : \mathcal{D} \times \mathcal{D} \to \mathbb{R}$:

$$\mathrm{div}_{\mathbf{x}}(v) = \sum_{\mathbf{z} \in \mathcal{D}} \big[v(\mathbf{x}, \mathbf{z}) - v(\mathbf{z}, \mathbf{x}) \big] \qquad \text{(net outgoing flux from } \mathbf{x}\text{)}$$

Set the probability flux $v(\mathbf{x}, \mathbf{z}) = p_t(\mathbf{x})\, R_t(\mathbf{x}, \mathbf{z})$. Then:

$$\begin{aligned}
\mathrm{div}_{\mathbf{x}}(p_t R_t) &= \sum_{\mathbf{z}} \big[p_t(\mathbf{x})\, R_t(\mathbf{x}, \mathbf{z}) - p_t(\mathbf{z})\, R_t(\mathbf{z}, \mathbf{x}) \big] \\
&= p_t(\mathbf{x}) \underbrace{\sum_{\mathbf{z}} R_t(\mathbf{x}, \mathbf{z})}_{=0 \text{ (rows sum to zero)}} - \sum_{\mathbf{z}} p_t(\mathbf{z})\, R_t(\mathbf{z}, \mathbf{x}) \\
&= -\sum_{\mathbf{z}} R_t(\mathbf{z}, \mathbf{x})\, p_t(\mathbf{z}) \;=\; -\frac{\partial}{\partial t} p_t(\mathbf{x})
\end{aligned}$$

Discrete Divergence (Figure)



Source: Gat et al., 2024

Conditional Rate Matrix

For our conditional path $p_t(x^i \mid \mathbf{x}_1) = (1-\kappa_t)\,\delta_{\mathtt{m}}(x^i) + \kappa_t\,\delta_{x_1^i}(x^i)$, the generating rate is (Theorem 3, Eq. 24 of Gat et al.):

$$\boxed{R_t^i(x^i, \mathbf{z} \mid \mathbf{x}_1) = \frac{\dot{\kappa}_t}{1-\kappa_t}\Big(\delta_{x_1^i}(x^i) - \delta_{z^i}(x^i)\Big)}$$

Reading the formula:

- If $z^i = \mathtt{m}$ and $x^i = x_1^i$: rate $= \frac{\dot{\kappa}_t}{1-\kappa_t} > 0$ (unmask towards target).
- If $z^i = x_1^i$: rate $= 0$ (already correct, do nothing).

Marginalisation of Conditional Rate Matrices

Theorem (Discrete analogue of Lipman et al.; Campbell et al.)
Given conditional rate matrices $R_t^i(x^i, \mathbf{z} \mid \mathbf{x}_1)$ that generate $p_t(\mathbf{x} \mid \mathbf{x}_1)$, the **marginal rate**

$$R_t^i(x^i, \mathbf{z}) = \sum_{\mathbf{x}_1 \in \mathcal{D}} R_t^i(x^i, \mathbf{z} \mid \mathbf{x}_1)\, p_t(\mathbf{x}_1 \mid \mathbf{z})$$

generates the marginal path $p_t(\mathbf{x})$.

Here $p_t(\mathbf{x}_1 \mid \mathbf{z}) = \frac{p_t(\mathbf{z}|\mathbf{x}_1)\, q(\mathbf{x}_1)}{p_t(\mathbf{z})}$ by Bayes' rule.

Same structure as the continuous case: conditional $\rightarrow$ marginal via posterior weighting.

Computing the Marginal Rate

Apply the marginalisation theorem to the conditional rate $R_t^i(x^i, \mathbf{z} \mid \mathbf{x}_1)$:

$$
\begin{aligned}
R_t^i(x^i, \mathbf{z}) &= \sum_{\mathbf{x}_1} R_t^i(x^i, \mathbf{z} \mid \mathbf{x}_1)\, p_t(\mathbf{x}_1 \mid \mathbf{z}) \\
&= \frac{\dot{\kappa}_t}{1-\kappa_t} \sum_{\mathbf{x}_1} \Big(\delta_{x_1^i}(x^i) - \delta_{z^i}(x^i) \Big) p_t(\mathbf{x}_1 \mid \mathbf{z}) \\
&= \frac{\dot{\kappa}_t}{1-\kappa_t} \Big(\underbrace{\textstyle\sum_{\mathbf{x}_1} \delta_{x_1^i}(x^i)\, p_t(\mathbf{x}_1 \mid \mathbf{z})}_{=:\, p_{1|t}(x^i|\mathbf{z})} - \delta_{z^i}(x^i) \Big)
\end{aligned}
$$

This gives the **marginal rate in denoiser form**:

$$
\boxed{R_t^i(x^i, \mathbf{z}) = \frac{\dot{\kappa}_t}{1-\kappa_t} \Big(p_{1|t}(x^i \mid \mathbf{z}) - \delta_{z^i}(x^i) \Big)}
$$

where $p_{1|t}(x^i \mid \mathbf{z})$ is the **probability denoiser**.

Training Objective

The marginal rate depends on $p_{1|t}(x^i \mid \mathbf{z})$ — the true posterior, which we don't have access to. **Parameterise** it with a neural network $p_{1|t}(x^i \mid \mathbf{z}; \theta)$ and train via **cross-entropy**:

$$\boxed{\mathcal{L}(\theta) = -\sum_{i=1}^{N} \mathbb{E}_{t,\mathbf{x}_1,X_t} \log p_{1|t}(x_1^i \mid X_t; \theta)}$$

Training procedure:

1. Draw $t \sim U[0,1]$, $\mathbf{x}_1 \sim q$.
2. Construct $X_t \sim p_t(\cdot \mid \mathbf{x}_1)$: independently set each token

$$X_t^i = \begin{cases} x_1^i & \text{w.p. } \kappa_t \\ \mathtt{m} & \text{w.p. } 1-\kappa_t \end{cases}$$

3. Feed X_t to the network, minimise $-\log p_{1|t}(x_1^i \mid X_t; \theta)$ over all positions i.

Sampling Algorithm

Algorithm 1: Discrete FM Sampling

Require: learned denoiser $p_{1|t}$, source sample $X \sim p$ (all masks), step size $h = 1/n$

---

for $t = 0, h, 2h, \ldots, 1 - h$ **do**

Compute rate: $R_t^i(x^i, X) = \dfrac{\dot{\kappa}_t}{1 - \kappa_t}\big(p_{1|t}(x^i \mid X; \theta) - \delta_X(x^i)\big)$

Sample each token: $X^i \sim \delta_{X^i}(\cdot) + h\, R_t^i(\cdot, X)$

end for

return X

Starting from all masks, we iteratively unmask tokens by sampling from the learned denoiser.

Continuous vs. Discrete Flow Matching

Component	Continuous FM	Discrete FM
State space	$\mathbb{R}^d$	$[V]^N$ (finite vocabulary)
Source (e.g.)	$\mathcal{N}(0, I)$	All-mask $\delta_{\mathbf{m}}$
"Flow"	ODE / deterministic flow	CTMC / stochastic jumps
"Velocity"	Vector field $v_t(\mathbf{x})$	Rate matrix $R_t^i(x^i, \mathbf{z})$
Generates p_t via	Continuity equation	Kolmogorov equation
Sampling	ODE solver	Euler CTMC updates

Comparison to Discrete Diffusion

Conceptual Difference

Discrete diffusion and **discrete flow models** both use CTMCs to move between noise and data:

	Discrete Diffusion	**Discrete Flow Models**
Choose first	Transition matrix Q_t	Probability path p_t
Then find	Marginals p_t via matrix exponentiation	Rate matrices R_t that generate p_t (via Kolmogorov eq.)

In flows, there may be **many valid rate matrices** that generate the desired path. This gives extra flexibility at inference time. The flow framework also makes it easier to design custom probability paths — e.g., the cubic scheduler used by Gat et al. (later).

Other Valid Rate Matrices

Suppose R_t is *some* valid rate matrix generating the conditional path $p_t(\cdot \mid \mathbf{x}_1)$. It is **not unique**.

A rate matrix R_t^{DB} is in **detailed balance** with $p_t(\cdot \mid \mathbf{x}_1)$ if

$$p_t(a \mid \mathbf{x}_1)\, R_t^{\mathrm{DB}}(a, b \mid \mathbf{x}_1) = p_t(b \mid \mathbf{x}_1)\, R_t^{\mathrm{DB}}(b, a \mid \mathbf{x}_1) \quad \forall\, a \neq b.$$

Detailed balance means flux is equal in both directions for all edges (a, b), so R_t^{DB} has *zero net effect* on p_t.

Therefore, for any $\eta \geq 0$,

$$R_t + \eta\, R_t^{\mathrm{DB}}$$

is **also a valid rate matrix** generating the same path $p_t(\cdot \mid \mathbf{x}_1)$.

The η Method: Controllable Stochasticity

For any $\eta \geq 0$, define

$$\boxed{R_t^\eta := R_t + \eta\, R_t^{\mathrm{DB}}}$$

All members of the family $\{R_t^\eta : \eta \geq 0\}$ generate the **same** conditional flow $p_t(\mathbf{x} \mid \mathbf{x}_1)$.

But the **sampling trajectory** changes:

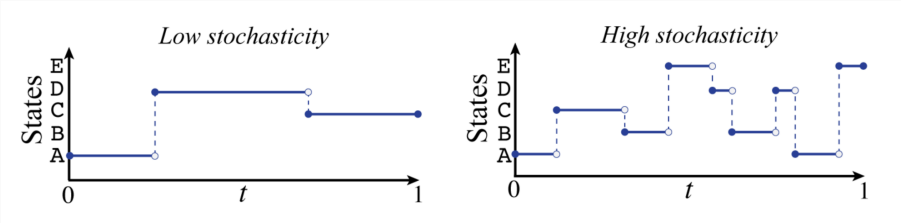
- $\eta > 0$**:** Extra jumps between states. E.g., unmasked tokens can become masked again.
- $\eta \gg 0$**:** More frequent jumps.

Crucially: η is chosen *at inference time* — the denoiser $p_{1|t}(x^i \mid \mathbf{z}; \theta)$ is trained once (via cross-entropy), and η is tuned afterwards.

Why Does $\eta > 0$ Help?

Intuition:

- With $\eta = 0$, once a token is unmasked to any error is frozen.
- With $\eta > 0$, tokens can be re-masked and re-generated.



Source: Campbell et al., 2024

Diffusion as a Special Case

Discrete diffusion (D3PM, absorbing state) corresponds to:

- The masking interpolant $p_t(x^i \mid \mathbf{x}_1) = (1 - \kappa_t)\,\delta_{\mathtt{m}}(x^i) + \kappa_t\,\delta_{x_1^i}(x^i)$.
- Sampling with $\eta = 0$ (minimum stochasticity = time-reversal).
- Using discrete time.

DFMs **generalise** by:

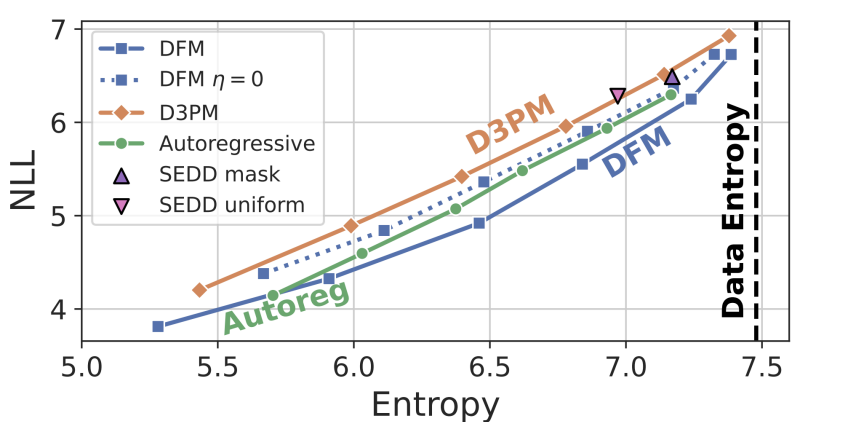
1. Providing a family $\{R_t^\eta : \eta \geq 0\}$ of valid sampling processes.
2. Choosing η at inference time, tuning stochasticity to the task.
3. Continuous time (but this is also possible for diffusion).

Empirical Results

η-Tuning on Text (Campbell et al.)

Setup: text8 (character-level English Wikipedia), 86M-param transformer.

Metric: NLL (via GPT-J-6B) vs. token entropy. Good samples = low NLL, entropy $\approx$ data. Swept over logit temperature.



η-Tuning on Text: Takeaways

- At $\eta = 0$, DFM performs **comparably to D3PM** – consistent with D3PM being the $\eta = 0$, discrete time special case.
- Increasing η improves the NLL–entropy frontier.
- **Optimal:** $\eta = 15$ — chosen *after* training by sweeping.
- Too much stochasticity ($\eta \gg 15$) degrades quality.

Practical Improvements (Gat et al.)

Gat et al. introduce two practical improvements within the same discrete flow framework:

1. Non-linear schedulers. Replace the linear scheduler $\kappa_t = t$ (used by Campbell et al.) with a tuned *cubic polynomial*. This controls how quickly tokens are unmasked and significantly affects generation quality.

2. Corrector sampling. Combine forward and backward probability velocities:

$$\bar{u}_t^i = \alpha_t \, \hat{u}_t^i - \beta_t \, \check{u}_t^i, \qquad \alpha_t - \beta_t = 1$$

This allows *re-masking* already-unmasked tokens and re-predicting them — conceptually similar to $\eta > 0$ stochasticity (extra jumps that let the model correct errors), but realised through the velocity field rather than detailed balance.

Unconditional Text Generation (Gat et al.)

150M-param models on OpenWebText, **no** corrector steps or temperature. Generative perplexity measured by Llama-2/3.

Method	**NFE**	**Llama-2 PPL** $\downarrow$	**Llama-3 PPL** $\downarrow$
D3PM (Austin et al.)	1024	—	—
SEDD (Lou et al.)	1024	27.2	43.9
Campbell et al. ($\kappa_t = t$)	1024	28.7	46.5
Gat et al. (mask interpolant)	1024	**22.5**	**33.8**
Gat et al. (uniform interpolant)	1024	**22.3**	**31.9**

Since no corrector steps are used here, the improvement over Campbell et al. comes mostly from the **cubic scheduler**.

What We Skipped: The General Framework

This presentation covers the absorbing state special case. The paper generalizes in two ways:

1. Arbitrary source–target couplings. Instead of independent $(X_0, X_1) \sim p(X_0)\, q(X_1)$, allow coupled (X_0, X_1). E.g., **conditional coupling**: randomly reveal some target tokens in the source,

$$X_0 = I \odot X_1 + (1 - I) \odot (\mathtt{m}, \ldots, \mathtt{m}), \qquad I \in \{0, 1\}^N.$$

2. General probability paths. Replace the two-component interpolant with an m-component mixture:

$$p_t(x^i \mid x_0, x_1) = \sum_{j=1}^{m} \omega_t^{i,j}\, w_j(x^i \mid x_0, x_1), \qquad \textstyle\sum_j \omega_t^{i,j} = 1.$$

Schedulers $\omega_t^{i,j}$ can vary per token and per component. E.g., a three-way mix of target, source, and uniform noise slightly improves perplexity.

References

- I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Q. Chen, G. Synnaeve, Y. Adi, Y. Lipman. *Discrete Flow Matching*, 2024. `arXiv:2407.15595`
- A. Campbell, J. Yim, R. Barzilay, T. Rainforth, T. Jaakkola. *Generative Flows on Discrete State-Spaces: Enabling Multimodal Flows with Applications to Protein Co-Design*, 2024. `arXiv:2402.04997`
- J. Austin, D. D. Johnson, J. Ho, D. Tarlow, R. van den Berg. *Structured Denoising Diffusion Models in Discrete State-Spaces*, 2021. `arXiv:2107.03006`
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. *Flow Matching for Generative Modeling*, 2022. `arXiv:2210.02747`